

실험 기록 설명서

무엇을 기록하고, 그 숫자로 무엇을 판단하는가

https://wandb.ai/DGIST_IRS/RadarAutonomous_FM

지표를 늘리는 것은 쉽고, 반년 뒤에 그 지표가 무엇이었는지 기억하는 것은 어렵습니다. 이 문서는 기록되는 값마다 어떻게 계산되고, 어떤 값이 정상이며, 벗어나면 무엇을 의심해야 하는지를 적습니다. 목록이 아니라 판독법입니다.

0. 한 장 요약

	무엇	왜 이것인가
학습	손실, 숫자 거리, 학습률, 처리량, 최고 메모리, 태스크 혼합	손실 하나로는 28개 태스크 중 무엇이 되고 있는지 알 수 없음
평가	태스크마다 두 벌 — 진짜 레이더와 남의 레이더	이 프로젝트의 질문은 정확도가 아니라 레이더가 쓰이는가
표본	생성 텍스트와 정답을 나란히	점수만으로는 틀린 답과 파싱 실패를 구별할 수 없음

모든 기록은 **rank 0** 만 합니다. 5개 프로세스가 같은 스칼라를 쓰면 처리량이 5배로 집계됩니다. 그리고 기록 호출은 실패해도 학습을 멈추지 않습니다 — 30시간짜리 실행이 대시보드 때문에 20시간째에 죽으면 안 되므로, 예외를 삼키고 이유를 한 번만 출력합니다.

RAVL_WANDB=off 로 끕니다.

1. 학습 중 기록

스텝 10 회마다 기록합니다. 여기서 스텝은 **옵티마이저가 한 번 밟히는 단위**이고, 그 안에 $\text{micro} \times \text{accum} \times \text{GPU수}$ 개의 샘플이 들어갑니다.

이름	계산	정상 범위 / 읽는 법
train/loss	정답 구간만의 크로스엔트로피. 프롬프트는 -100 으로 마스킹되어 손실에 들어가지 않음. 기울기 누적으로 나눈 것을 다시 곱해 원래 크기로 기록	원본에서 시작하면 4.5 부근, 이어받으면 0.3~0.4. 태스크가 28종이라 배치 구성에 따라 ± 0.06 흔들림 — 그보다 작은 변화는 추세가 아님
train/number_distance	숫자 토큰 2,503개에 준 확률로 기댓값을 만들고 정답 값과의 smooth L1. 배치의 값 크기로 나눠 정규화	"몇 미터 틀렸나"에 가까운 양. 크로스엔트로피가 못 보는 것을 봄. 손실은 내려가는데 이것이 안 내려가면 형식만 배우고 값은 못 맞히는 중
train/radar_hinge	남의 레이더를 넣었을 때 손실이 margin 만큼 나빠지지 않으면 그 부족분	--radar-contrast 를 컷을 때만 나옴. 0 이면 레이더를 안 읽고 있다는 뜻
train/lr	스케줄러의 현재 학습률	워밍업과 감쇠가 의도대로인지. 재개 후 갑자기 튀면 스케줄이 처음부터 다시 시작한 것
perf/sample_per_s	누적 샘플 ÷ 경과 시간	5 × B200 에서 17~18. 떨어지면 병목 — 다만 micro 를 바꿔도 거의 안 변함(측정: 8/16/24 에서 17.5/17.1/18.3)
perf/peak_gib	<code>torch.cuda.max_memory_allocated()</code>	한계 178.4. 계단식으로 오르는 중이면 아직 최악의 배치를 안 만난 것 이고 OOM 위험이 남아 있음. 평평해지면 안전
perf/samples_seen	$\text{step} \times \text{micro} \times \text{world} \times \text{accum}$	전체 3,343만 대비 진행률. 200만이면 6%

1.1 시작할 때 config 에 박히는 것

항목	내용
실행 인자 전부	<code>micro_batch</code> , <code>accum</code> , <code>lr</code> , <code>seed</code> , <code>digit_weight</code> , <code>resume</code> ... 나중에 두 실행을 비교할 때 무엇이 달랐는지가 여기 남음
<code>global_batch</code>	$\text{micro} \times \text{accum} \times \text{world}$. 학습 결과에 영향을 주는 것은 이 곱 하나이고 micro 와 accum 을 어떻게 나눴는지는 무관
<code>vocab</code> , <code>trainable_params</code>	숫자 토큰 추가 후 154,165 / 8,820.0 M. 어휘가 바뀌면 이전 체크포인트와 이어붙일 수 없으므로 기록해 둠
<code>mix/⟨task⟩</code> 28개	실제로 뽑힌 태스크 비율. 혼합 가중치가 의도대로 반영됐는지 여기서만 보임 — 이름이 바뀐 가중치 키가 조용히 무시된 적이 있음

종료 시에는 `final/step`, `final/loss`, `final/samples`, `final/hours`, `final/out` 이 요약에 남습니다. `final/out` 은 체크포인트 경로라, 대시보드의 실행과 디스크의 가중치를 잇는 유일한 끈입니다.

2. 평가 기록

평가는 태스크마다 두 벌을 냅니다. 이것이 이 프로젝트의 중심 설계입니다.

	무엇	의미
full/<task>/...	그 클립의 진짜 레이더로 생성	실제 성능
shuffled/<task>/...	다른 클립의 레이더를 끼워 넣고 생성	레이더가 없는 것과 마찬가지로 조건
둘의 차이	= 레이더 기여도	0에 가까우면 그 태스크는 카메라와 ego만으로 풀리고 있음. 정확도가 높아도 이 차이가 0이면 레이더 모델이라 부를 수 없음

2.1 태스크 종류마다 다른 지표

태스크	지표	주의
검출 · 이동판정	f1, precision, recall, class_acc, range_mae, az_mae (3D는 size_mae, yaw_mae 추가)	매칭은 거리 2 m 또는 IoU 0.3. 매칭된 것만 오차 평균에 들어가므로 F1이 낮으면 range_mae가 작으면 '우연히 가까운 것끼리만 맞은' 상태
추적	f1, class_acc, id_carried	id_carried는 이력에 있던 물체가 같은 번호를 유지했는가. 한 스텝만 생성하면 이력이 정답이라 판정 대상이 없어 비어 있음
자차 경로	displacement_mae_m / speed_mae_ms, yaw_mae_degs, coverage	coverage는 요구한 지평선 중 몇 개를 답했는가. 1.00이 아니면 형식이 깨진 것
물체 미래	range_mae_m, az_mae_deg, coverage	커버리지 0.00은 점수가 아니라 파싱 실패 . 실제로 그런 적이 있음
QA	accuracy	5지선다이므로 우연이 0.20
프로브	parsed, rel_err	답 자체가 수치인 태스크
CoT 전부	위에 더해 cot_parsed	근거 형식을 지킨 비율. 낮으면 점수가 낮은 이유가 능력이 아니라 형식

2.2 samples 표 — 점수 옆에 실제 텍스트

태스크 · 생성 · 정답 세 열입니다. 점수만으로는 틀린 답과 파싱 실패를 구별할 수 없습니다. 실제로 agent_traj_bbox의 커버리지 0.00은 모델이 못 맞힌 것이 아니라 채점기가 못 읽은 것이었고, 표를 봐야 드러나는 종류였습니다.

3. 겹쳐 봐야 보이는 것

겹쳐 볼 것	무엇이 보이냐
train/loss + train/number_distance	손실은 내려가는데 숫자 거리가 평평한 구간 = 형식은 배우고 값은 못 맞히는 중. 숫자가 정답 토큰의 44~88% 를 차지하므로 이 둘이 갈라지면 '문장은 그럴듯한데 수치가 틀린' 모델이 되고 있음
full/<task>/f1 + shuffled/<task>/f1	태스크별 레이더 기여도. 두 선이 붙어 있으면 그 태스크는 레이더 없이 풀리고 있고, 그 태스크의 점수를 올려도 이 프로젝트의 주장은 강해지지 않음
perf/peak_gib 추세	평평하면 안전, 계단식으로 오르면 아직 최악의 배치를 안 만난 것. 실제로 평균 121 GiB 에서 안정적으로 보이던 실행이 6시간 반 뒤 37 GiB 짜리 일시 할당을 만나 죽었음 — 평균이 아니라 꼬리를 봐야 함
mix/<task> 와 태스크별 점수	혼합에서 비중이 큰데 점수가 안 오르는 태스크는 데이터가 아니라 정의를 의심할 자리

4. 기록하지 않는 것과 그 이유

	이유
검증 손실	모델 선택을 test 에서 하고 있어 val 을 train 에 합쳤음. 지금 구조에서 검증 손실은 학습 손실의 복사본에 가까움
기울기 노름 · 가중치 히스토그램	값이 크지만 해석이 어렵고, 지금 겪는 실패(형식 붕괴, OOM, 채점기 결함)는 전부 다른 곳에서 드러났음. 필요해지면 그때 추가
모든 스텝	10 스텝마다 기록. 15초/스텝이므로 2.5분 간격이고, 그보다 촘촘해도 배치 간 변동에 묻힘
생성 텍스트 전량	대시보드에는 표본만. 전량은 runs/12_eval/*.generations.jsonl 에 남고 git 에는 올리지 않음

5. 다루는 법

```
wandb login # 한 번만
RAVL_WANDB=off torchrun ... # 이번 실행만 기록 끄기
WANDB_MODE=offline torchrun ... # 로컬에 쌓고 나중에 wandb sync
RAVL_WANDB_PROJECT=other ... # 다른 프로젝트로
```

실행 이름은 언제 돌렸는지가 아니라 **무엇이 달랐는**지로 만들어집니다 — sft-8B-full-b54-lr0.00015-num0.3-s0-0807-2048. 시드만 다른 두 실행이 나란히 경렬되고, 배치나 목적함수가 다른 실행은 목록에서 바로 구별됩니다.

문서 생성: `python -m datatools.build_tracking_report`